EXPERIMENT-1:

Data Exploration and Cleaning with pandas

Code:

student_marks.csv

```
reg_no,department,gender,attendance,internal,final_exam,study_hours
101,CSE,Female,85,45,88,5
102,ECE,Male,90,42,78,4
103,CSE,Female,95,48,92,7
104,IT,Male,80,40,75,3
105,ECE,Female,88,44,85,6
106,CSE,Male,,39,70,4
107,IT,Female,75,35,65,2
108,CSE,Female,140,47,95,8
109,ECE,Male,92,,82,5
110,IT,Female,85,41,,3
111,CSE,Male,90,46,89,
112,ECE,Female,88,43,84,6
```

experiment1.py

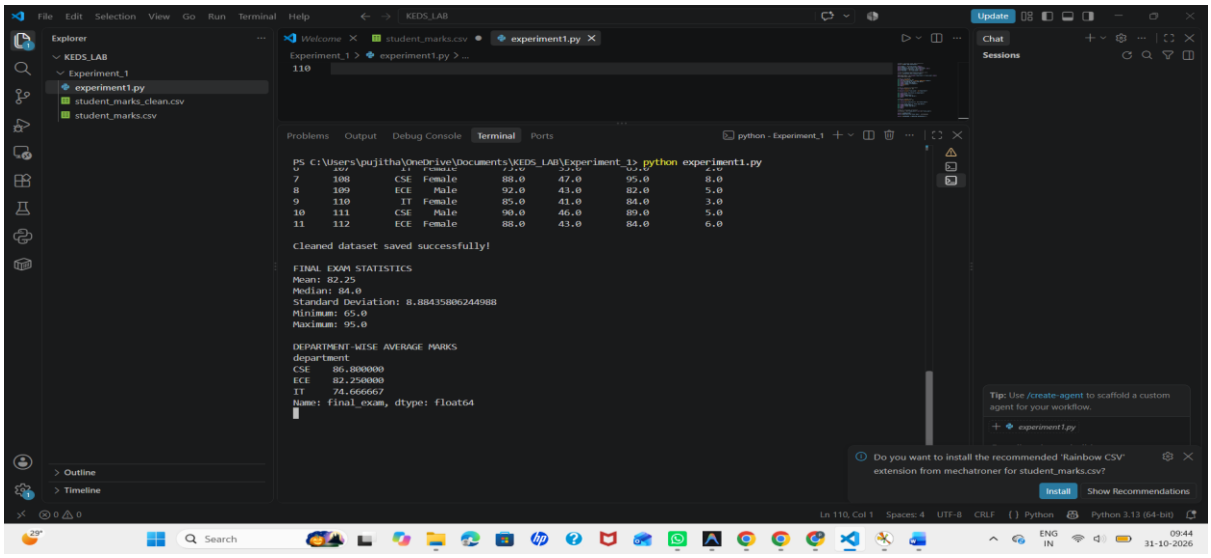
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
# Step 1: Load the dataset
df = pd.read_csv("student_marks.csv")
print("ORIGINAL DATASET")
print(df)
# Step 2: Display dataset information
print("\nFIRST FIVE RECORDS")
print(df.head())
print("\nDATASET SHAPE")
```

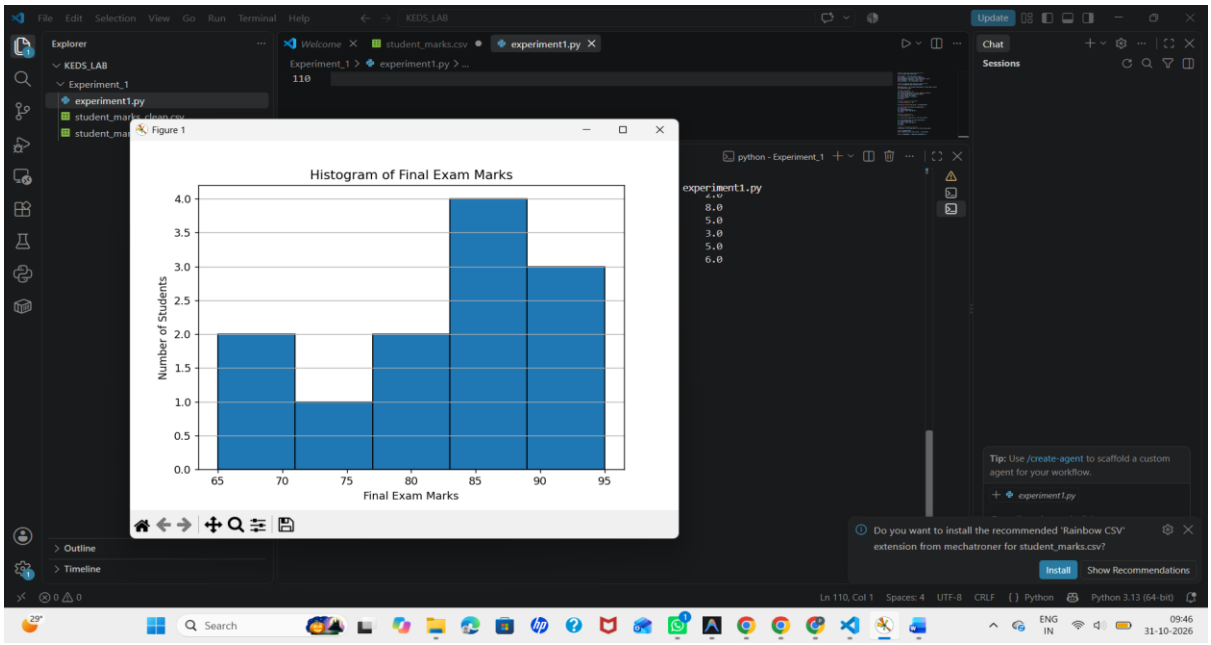
```
print(df.shape)
print("\nDATASET INFORMATION")
df.info()
print("\nDESCRIPTIVE STATISTICS")
print(df.describe())
# Step 3: Find missing values
print("\nMISSING VALUES")
print(df.isna().sum())
# Step 4: Handle impossible attendance values
df.loc[df["attendance"] > 100, "attendance"] = np.nan
# Step 5: Fill missing numerical values using median
numeric_columns = [
    "attendance", "internal", "final_exam", "study_hours"
]
for col in numeric_columns:
    df[col] = df[col].fillna(df[col].median())
# Step 6: Fill missing categorical values using mode
categorical_columns = ["department", "gender"]
for col in categorical_columns:
    df[col] = df[col].fillna(df[col].mode()[0])
# Step 7: Remove duplicate records
df = df.drop_duplicates()
print("\nCLEANED DATASET")
print(df)
# Step 8: Save cleaned dataset
df.to_csv("student_marks_clean.csv", index=False)
print("\nCleaned dataset saved successfully!")
# Step 9: Calculate final exam statistics
print("\nFINAL EXAM STATISTICS")
print("Mean:", df["final_exam"].mean())
```

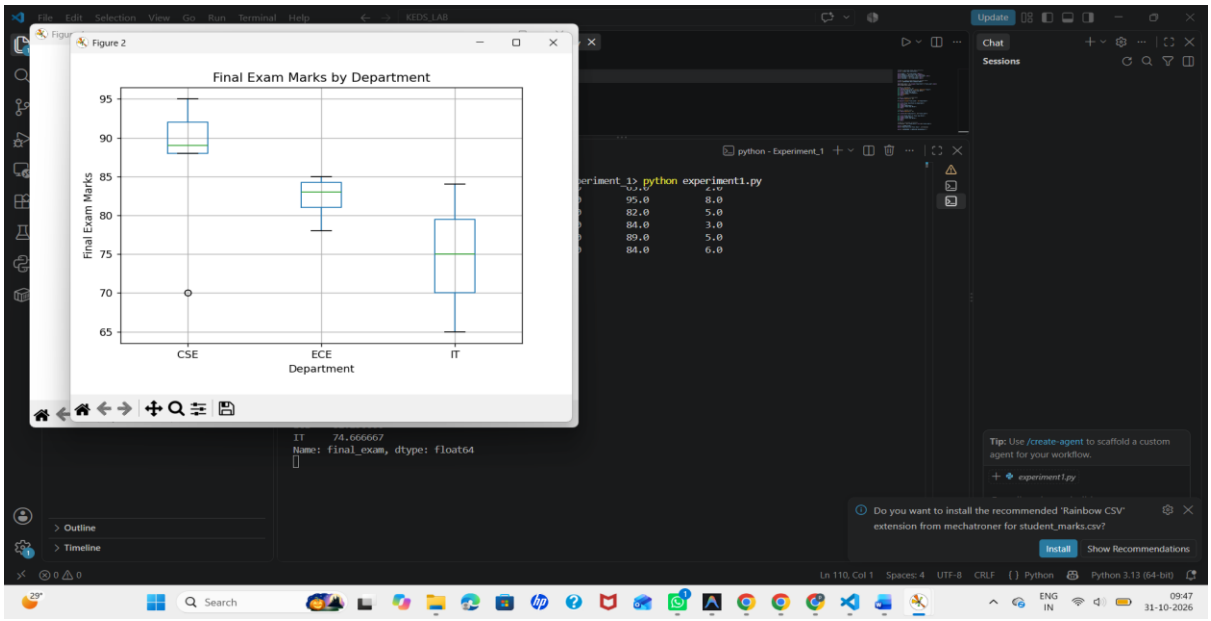
```
print("Median:", df["final_exam"].median())
print("Standard Deviation:", df["final_exam"].std())
print("Minimum:", df["final_exam"].min())
print("Maximum:", df["final_exam"].max())
# Step 10: Compare department-wise average marks
print("\nDEPARTMENT-WISE AVERAGE MARKS")
department_mean = df.groupby("department")["final_exam"].mean()
print(department_mean)
# Step 11: Histogram
plt.figure(figsize=(7, 5))
plt.hist(df["final_exam"], bins=5, edgecolor="black")
plt.title("Histogram of Final Exam Marks")
plt.xlabel("Final Exam Marks")
plt.ylabel("Number of Students")
plt.grid(axis="y")
plt.show()
# Step 12: Boxplot by department
plt.figure(figsize=(7, 5))
df.boxplot(column="final_exam", by="department")
plt.title("Final Exam Marks by Department")
plt.suptitle("")
plt.xlabel("Department")
plt.ylabel("Final Exam Marks")
plt.show()
# Step 13: Scatter plot
plt.figure(figsize=(7, 5))
plt.scatter(df["study_hours"], df["final_exam"])
plt.title("Study Hours vs Final Exam Marks")
plt.xlabel("Study Hours")
plt.ylabel("Final Exam Marks")
```

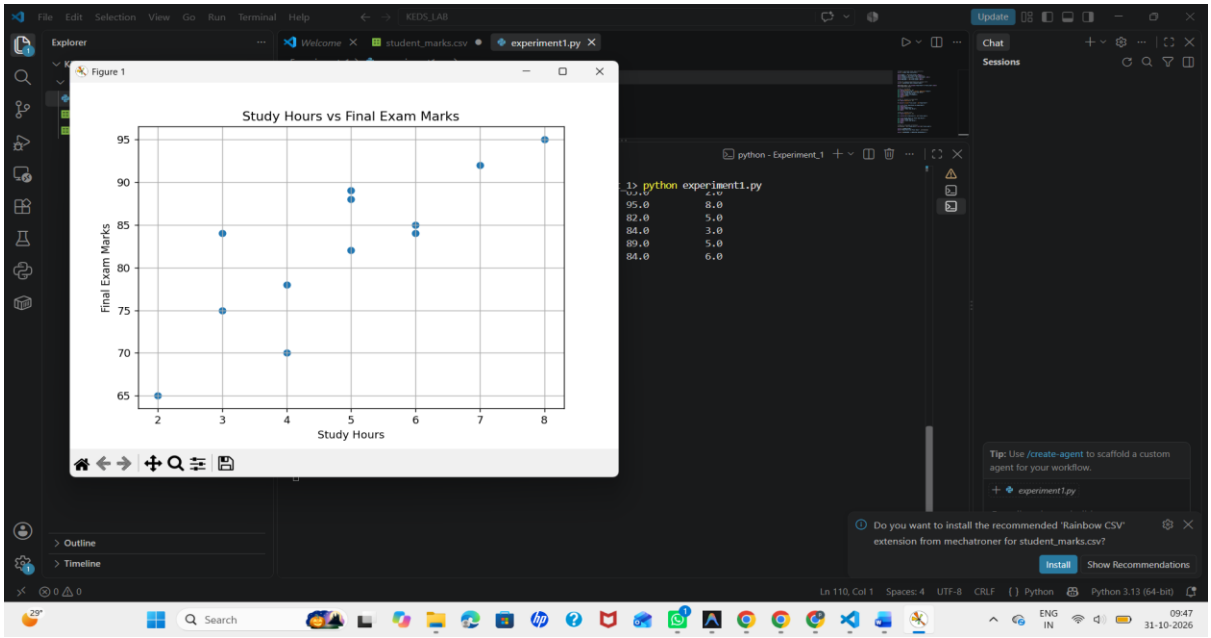
```
plt.grid()
plt.show()
# Step 14: Calculate correlation
correlation = df["study_hours"].corr(df["final_exam"])
print("\nCORRELATION")
print("Study Hours and Final Exam:", correlation)
print("\nEXPERIMENT 1 COMPLETED SUCCESSFULLY!")
```

OUTPUT:






Figure 2
Final Exam Marks by Department
Final Exam Marks
Department
CSE
ECE
IT
95
90
85
80
75
70
65
python experiment1.py
95.0 8.0
82.0 5.0
84.0 3.0
89.0 5.0
84.0 6.0
IT 74.666667
Name: final_exam, dtype: float64
Do you want to install the recommended 'Rainbow CSV' extension from mechatroner for student_marks.csv?
Install
Show Recommendations


Figure 1
Study Hours vs Final Exam Marks
Final Exam Marks
Study Hours
95
90
85
80
75
70
65
2
3
4
5
6
7
8
python experiment1.py
95.0 8.0
82.0 5.0
84.0 3.0
89.0 5.0
84.0 6.0
Do you want to install the recommended 'Rainbow CSV' extension from mechatroner for student_marks.csv?
Install
Show Recommendations

Experiment 2: Semantic Network and Inheritance Queries

Program:

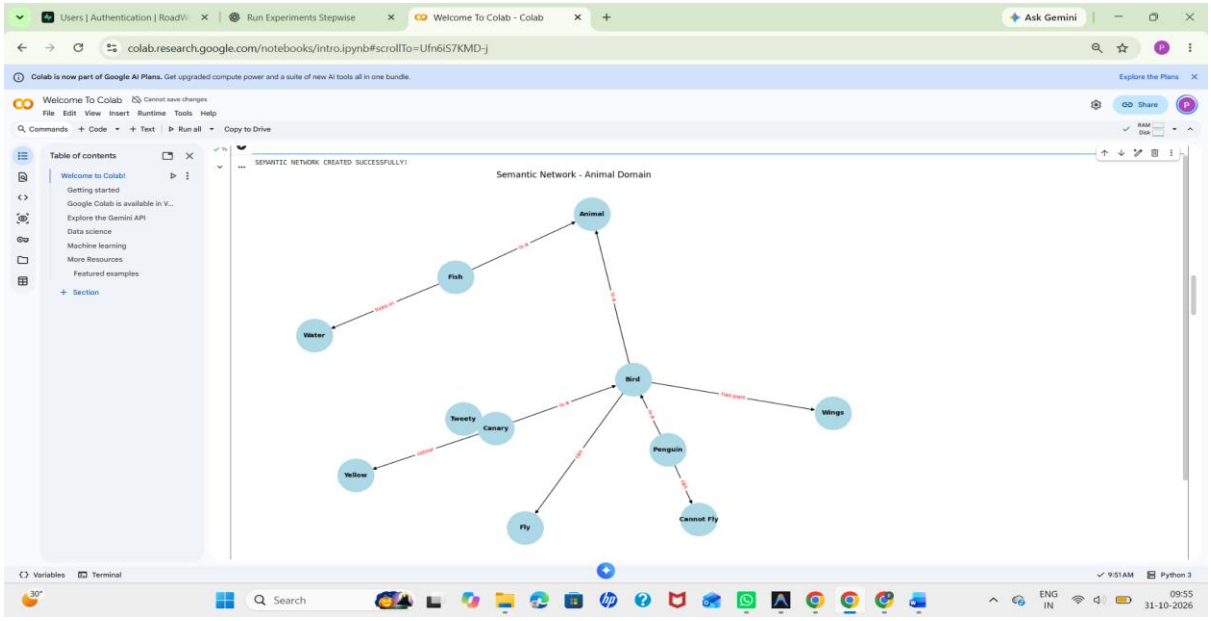
```
!pip install networkx matplotlib -q
import networkx as nx
import matplotlib.pyplot as plt
# Step 1: Create a directed graph
G = nx.DiGraph()
# Step 2: Add nodes
nodes = [
    "Animal", "Bird", "Fish", "Canary",
    "Penguin", "Tweety", "Wings",
    "Yellow", "Water", "Fly", "Cannot Fly"
]
G.add_nodes_from(nodes)
# Step 3: Add relationships
G.add_edge("Bird", "Animal", relation="is-a")
G.add_edge("Fish", "Animal", relation="is-a")
G.add_edge("Canary", "Bird", relation="is-a")
G.add_edge("Penguin", "Bird", relation="is-a")
G.add_edge("Tweety", "Canary", relation="instance-of")
# Step 4: Add properties
G.add_edge("Bird", "Wings", relation="has-part")
G.add_edge("Bird", "Fly", relation="can")
G.add_edge("Fish", "Water", relation="lives-in")
G.add_edge("Canary", "Yellow", relation="colour")
# Penguin overrides the inherited flying property
G.add_edge("Penguin", "Cannot Fly", relation="can")
print("SEMANTIC NETWORK CREATED SUCCESSFULLY!")
# Step 5: Draw semantic network
```

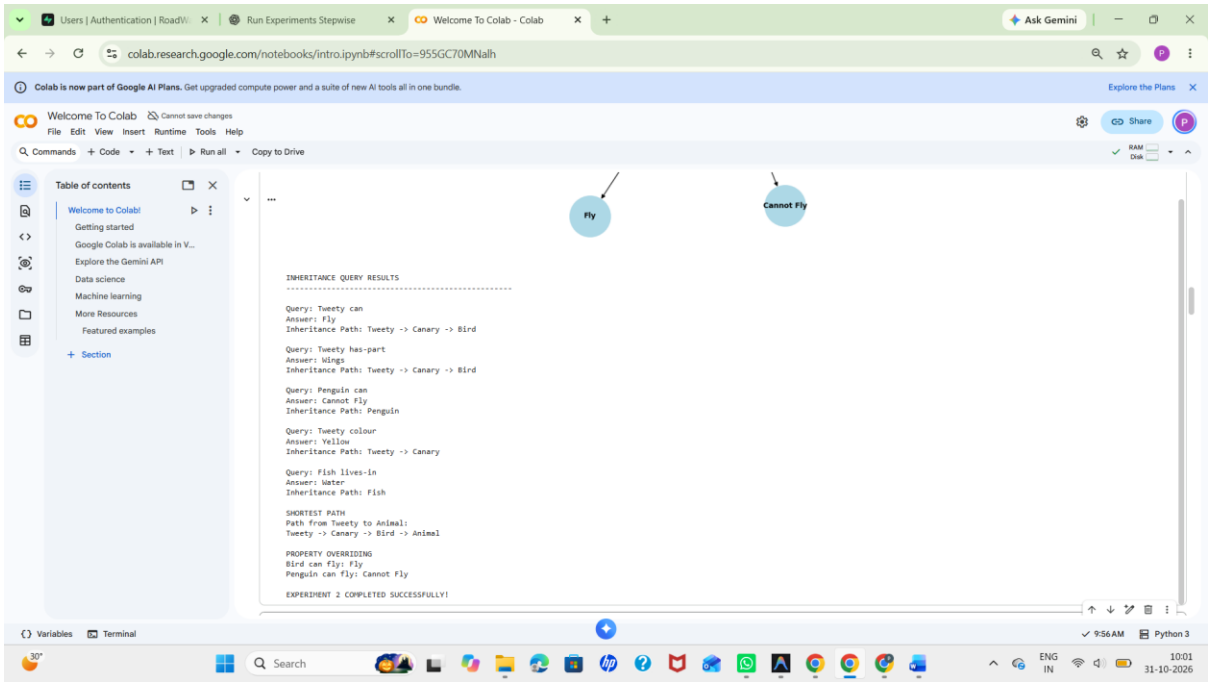
```
plt.figure(figsize=(12, 8))
pos = nx.spring_layout(G, seed=42)
nx.draw(
    G, pos,
    with_labels=True,
    node_color="lightblue",
    node_size=2500,
    font_size=9,
    font_weight="bold",
    arrows=True
)
edge_labels = nx.get_edge_attributes(G, "relation")
nx.draw_networkx_edge_labels(
    G, pos,
    edge_labels=edge_labels,
    font_color="red",
    font_size=8
)
plt.title("Semantic Network - Animal Domain")
plt.savefig("semantic_network.png", dpi=300, bbox_inches="tight")
plt.show()
# Step 6: Inheritance function
def inherits(node, relation):
    visited = set()
    queue = [(node, [node])]
while queue:
        current, path = queue.pop(0)
        if current in visited:
            continue
```

```
        visited.add(current)
        # Check local properties first
        for _, target, data in G.out_edges(current, data=True):
            if data.get("relation") == relation:
                return target, path
        # Follow parent relationships
        for parent in G.successors(current):
            edge_relation = G[current][parent]["relation"
            if edge_relation in ["is-a", "instance-of"]:
                queue.append((parent, path + [parent]))
    return None, []
# Step 7: Test five inheritance queries
queries = [
    ("Tweety", "can"),
    ("Tweety", "has-part"),
    ("Penguin", "can"),
    ("Tweety", "colour"),
    ("Fish", "lives-in")
]
print("\nINHERITANCE QUERY RESULTS")
print("-" * 50)
for node, relation in queries:
    result, path = inherits(node, relation)
    print("\nQuery:", node, relation)
    if result:
        print("Answer:", result)
        print("Inheritance Path:", " -> ".join(path))
    else:
        print("Answer: Property not found")
```

```
# Step 8: Find shortest path
print("\nSHORTEST PATH")
try:
    path = nx.shortest_path(
        G,
        source="Tweety",
        target="Animal"
    )
    print("Path from Tweety to Animal:")
    print(" -> ".join(path))
except nx.NetworkXNoPath:
    print("No path found!")
# Step 9: Demonstrate property overriding
print("\nPROPERTY OVERRIDING")
print("Bird can fly:", inherits("Bird", "can")[0])
print("Penguin can fly:", inherits("Penguin", "can")[0])
print("\nEXPERIMENT 2 COMPLETED SUCCESSFULLY!")
```

OUTPUT:





Experiment 3: Frames with Defaults, Inheritance and If-Needed Procedures

Program:

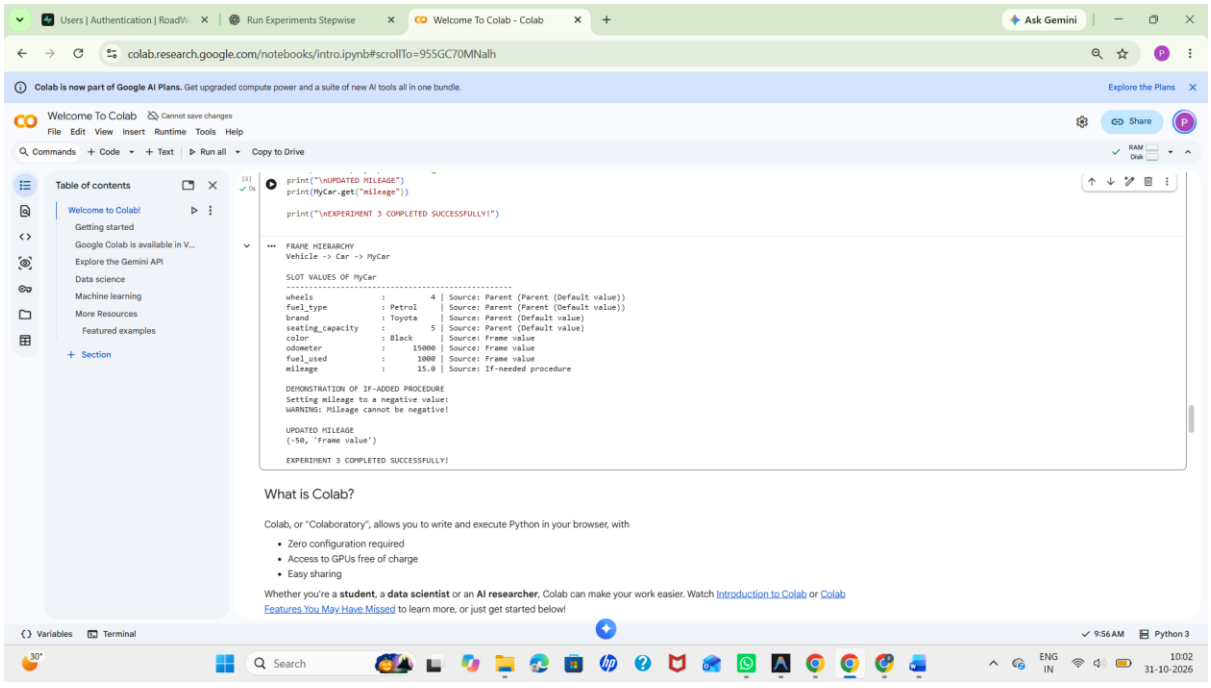
```
# Step 1: Create Frame class
class Frame:
    def __init__(self, name, parent=None):
        self.name = name
        self.parent = parent
        self.slots = {}
    # Step 2: Add slots
    def add_slot(self, slot, value=None, default=None,
                 if_needed=None, if_added=None):
        self.slots[slot] = {
            "value": value,
            "default": default,
            "if_needed": if_needed,
            "if_added": if_added
        }
    # Step 3: Get slot value using inheritance
```

```
    def get(self, slot):
        if slot in self.slots:
            data = self.slots[slot]
            if data["value"] is not None:
              return data["value"], "Frame value"
            if data["if_needed"] is not None:
                return data["if_needed"](self), "If-needed procedure"
            if data["default"] is not None:
                return data["default"], "Default value"
        if self.parent is not None:
            value, source = self.parent.get(slot)
            return value, "Parent (" + source + ")"
        return None, "Not found
    # Step 4: Set slot value and execute demon
    def set(self, slot, value):
        if slot not in self.slots:
            self.add_slot(slot
        self.slots[slot]["value"] = value
        if self.slots[slot]["if_added"] is not None:
            self.slots[slot]["if_added"](value)
# Step 5: Define warning procedure
def mileage_warning(value):
    if value < 0:
        print("WARNING: Mileage cannot be negative!")
# Step 6: Define if-needed procedure
def calculate_mileage(frame):
    odometer, _ = frame.get("odometer")
    fuel_used, _ = frame.get("fuel_used")
    if fuel_used == 0:
        return
```

```
    return odometer / fuel_used
# Step 7: Create Vehicle frame
Vehicle = Frame("Vehicle")
Vehicle.add_slot("wheels", default=4)
Vehicle.add_slot("fuel_type", default="Petrol")
# Step 8: Create Car frame
Car = Frame("Car", parent=Vehicle)
Car.add_slot("brand", default="Toyota")
Car.add_slot("seating_capacity", default=5)
# Step 9: Create MyCar frame
MyCar = Frame("MyCar", parent=Car)
MyCar.add_slot("color", value="Black")
MyCar.add_slot("odometer", value=15000)
MyCar.add_slot("fuel_used", value=1000)
MyCar.add_slot(
    "mileage",
    if_needed=calculate_mileage,
    if_added=mileage_warning
)
# Step 10: Display frame details
print("FRAME HIERARCHY")
print("Vehicle -> Car -> MyCar")
print("\nSLOT VALUES OF MyCar")
print("-" * 50
slots = [
    "wheels",
    "fuel_type",
    "brand",
    "seating_capacity",
    "color",
```

```
    "odometer",
    "fuel_used",
    "mileage"
]
for slot in slots:
    value, source = MyCar.get(slot)
    print(f"{slot:20} : {value:10} | Source: {source}")
# Step 11: Demonstrate setting a slot
print("\nDEMONSTRATION OF IF-ADDED PROCEDURE")
print("Setting mileage to a negative value:")
MyCar.set("mileage", -50)
# Step 12: Display updated mileage
print("\nUPDATED MILEAGE")
print(MyCar.get("mileage"))
print("\nEXPERIMENT 3 COMPLETED SUCCESSFULLY!")
```

OUTPUT:



Experiment 4: Rule-Based System with Forward Chaining

Program:

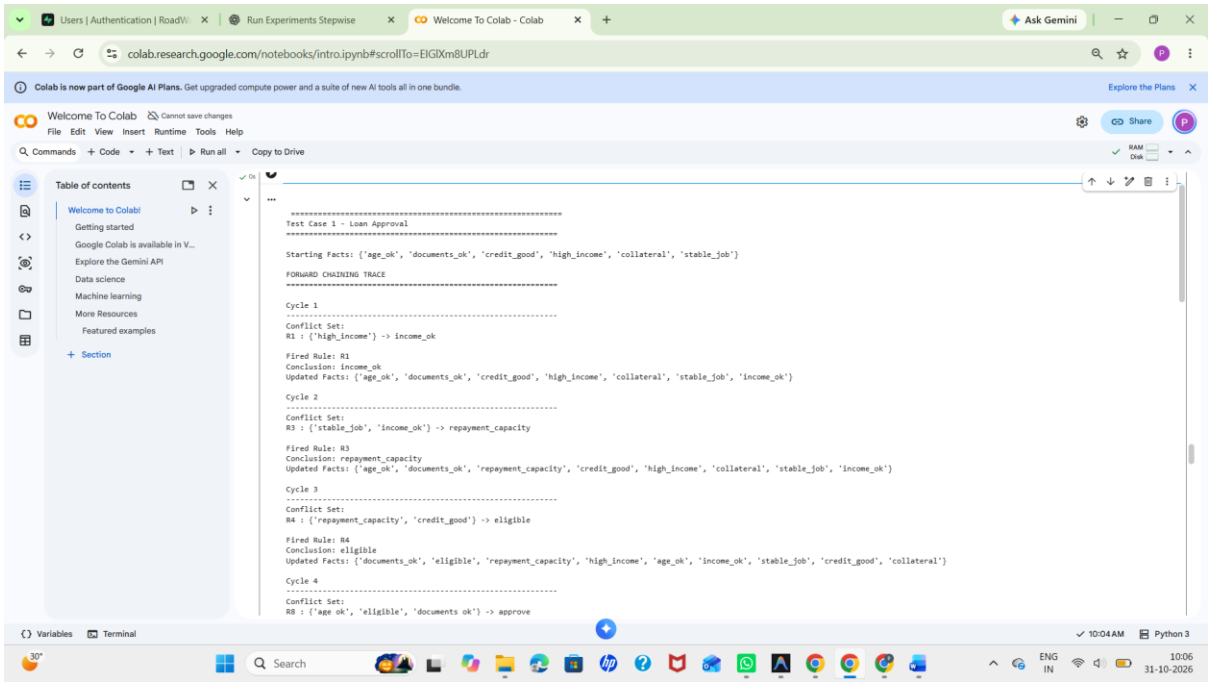
```
rules = [
    ("R1", {"high_income"}, "income_ok"),
    ("R2", {"low_income"}, "income_low"),
    ("R3", {"income_ok", "stable_job"}, "repayment_capacity"),
    ("R4", {"credit_good", "repayment_capacity"}, "eligible"),
    ("R5", {"existing_debt"}, "debt_risk"),
    ("R6", {"debt_risk", "high_debt"}, "reject"),
    ("R7", {"income_low"}, "reject"),
    ("R8", {"eligible", "documents_ok", "age_ok"}, "approve"),
    ("R9", {"eligible", "collateral"}, "approve_secured"),
    ("R10", {"credit_bad"}, "reject")
]
# Step 2: Forward chaining function
def forward_chaining(initial_facts, strategy="first"):
    facts = set(initial_facts)
    cycle = 1
    print("\nStarting Facts:", facts)
    print("\nFORWARD CHAINING TRACE")
    print("=" * 60)
    while True:
        # Find all matching rules
        conflict_set = [
            rule for rule in rules
            if rule[1].issubset(facts)
            and rule[2] not in facts
        ]
```

```
    # Stop if no rules match
    if not conflict_set:
        print("\nNo more rules can be fired.")
        break
    print(f"\nCycle {cycle}")
    print("-" * 60)
    print("Conflict Set:")
    for rule in conflict_set:
        print(rule[0], ":", rule[1], "->", rule[2])
    # Step 3: Conflict resolution
    if strategy == "first":
        selected_rule = conflict_set[0]
    elif strategy == "specific":
        selected_rule = max(
            conflict_set,
            key=lambda rule: len(rule[1])
        )
    # Step 4: Fire selected rule
    rule_id, conditions, conclusion = selected_rule
    print("\nFired Rule:", rule_id)
    print("Conclusion:", conclusion)
    facts.add(conclusion
    print("Updated Facts:", facts)
    cycle += 1
print("\nFINAL FACTS:", facts
# Display final decision
if "approve" in facts:
    print("FINAL DECISION: LOAN APPROVED")
elif "approve_secured" in facts:
    print("FINAL DECISION: SECURED LOAN APPROVED")
```

```
    elif "reject" in facts:
        print("FINAL DECISION: LOAN REJECTED")
    else:
        print("FINAL DECISION: NO CONCLUSION")
    return facts
# Step 5: Three test cases
test_cases = {
    "Test Case 1 - Loan Approval": {
        "high_income",
        "stable_job",
        "credit_good",
        "documents_ok",
        "age_ok",
        "collateral"
    },
    "Test Case 2 - Loan Rejection": {
        "low_income"
    }
    "Test Case 3 - No Conclusion": {
        "student",
        "part_time",
        "unknown_status"
    }
}
# Step 6: Execute three test cases
for name, facts in test_cases.items():
    print("\n\n", "=" * 60)
    print(name)
    print("=" * 60)
    forward_chaining(facts, strategy="first")
```

```
# Step 7: Compare conflict resolution strategies
print("\n\nCONFLICT RESOLUTION COMPARISON")
print("=" * 60)
comparison_facts = {
    "high_income",
    "stable_job",
    "credit_good",
    "documents_ok",
    "age_ok",
    "collateral",
    "existing_debt",
    "high_debt"
}
print("\nStrategy 1: First Matching Rule")
result1 = forward_chaining(
    comparison_facts,
    strategy="first"
)
print("\n\nStrategy 2: Most Specific Rule First")
result2 = forward_chaining(
    comparison_facts,
    strategy="specific"
)
print("\n\nCOMPARISON RESULT")
print("=" * 60)
print("First Matching Rule:", result1)
print("Most Specific Rule:", result2)
print("\nEXPERIMENT 4 COMPLETED SUCCESSFULLY!")
```

OUTPUT:



Experiment 5: Backward Chaining with a Why/How Explanation Facility

Program:

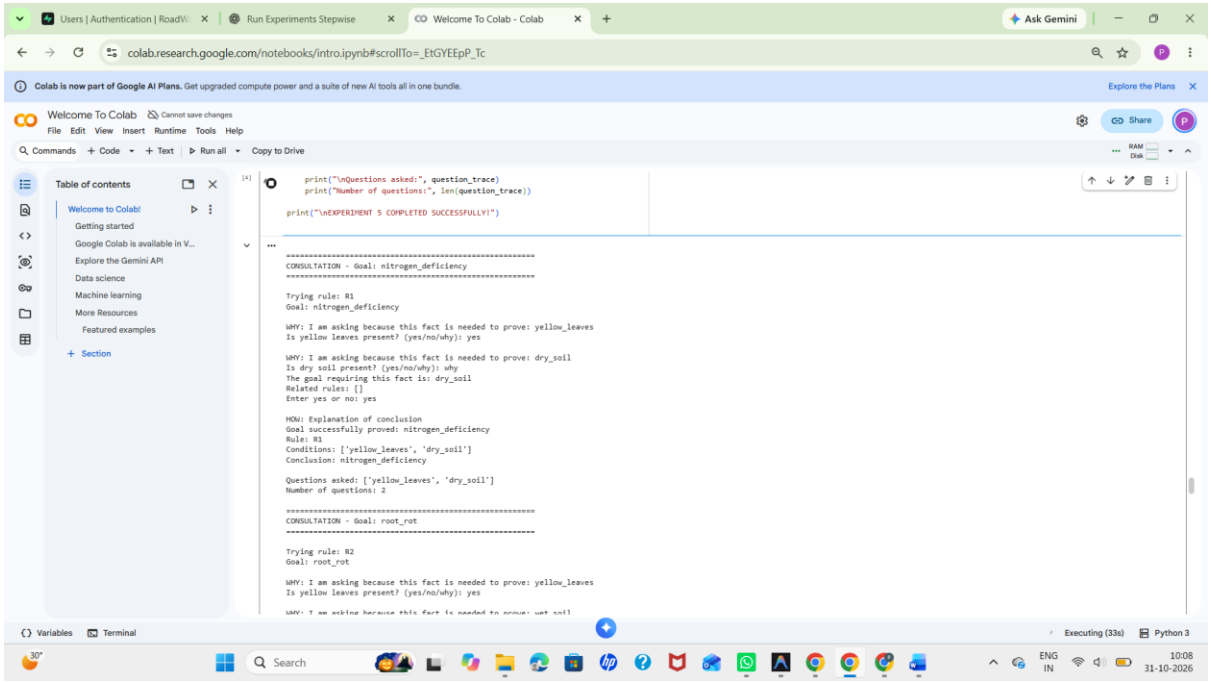
```
# Step 1: Define rules
rules = [
    ("R1", ["yellow_leaves", "dry_soil"], "nitrogen_deficiency"),
    ("R2", ["yellow_leaves", "wet_soil"], "root_rot"),
    ("R3", ["brown_spots", "humid_weather"], "fungal_infection"),
    ("R4", ["fungal_infection", "leaf_damage"], "leaf_blight"),
    ("R5", ["white_powder", "dry_weather"], "powdery_mildew"),
    ("R6", ["root_rot", "wilting"], "root_disease"),
    ("R7", ["nitrogen_deficiency", "slow_growth"], "nutrient_problem"),
    ("R8", ["nutrient_problem"], "fertilizer_needed"),
    ("R9", ["leaf_blight", "humid_weather"], "fungicide_needed"),
    ("R10", ["powdery_mildew"], "fungicide_needed"),
    ("R11", ["root_disease"], "root_treatment_needed"),
```

```
    ("R12", ["brown_spots", "leaf_damage"], "leaf_disease")
]
# Step 2: Askable facts
askable = [
    "yellow_leaves", "dry_soil", "wet_soil",
    "brown_spots", "humid_weather", "leaf_damage",
    "white_powder", "dry_weather", "wilting",
    "slow_growth"
]
# Step 3: Store facts and explanation
facts = {}
proof_trace = []
question_trace = []
def prove(goal, depth=0)
    if goal in facts:
        return facts[goal]
    applicable_rules = [
        rule for rule in rules if rule[2] == goal
    if not applicable_rules:
        if goal not in askable:
            return False
        print("\nWHY: I am asking because this fact is needed to prove:", goal)
        answer = input(
            "Is " + goal.replace("_", " ") +
            " present? (yes/no/why): "
        ).lower().strip(
        if answer == "why":
            print("The goal requiring this fact is:", goal)
            print("Related rules:", applicable_rules)
            answer = input("Enter yes or no: ").lower().strip(
```

```
        value = answer in ["yes", "y"
        facts[goal] = value
        question_trace.append(goal
        return valu
    for rule_id, conditions, conclusion in applicable_rules
        print("\nTrying rule:", rule_id)
        print("Goal:", goal)
        all_proved = True
        for condition in conditions:
            if not prove(condition, depth + 1):
                all_proved = False
                brea
        if all_proved:
            facts[goal] = True
            proof_trace.append(
                (rule_id, conditions, conclusion)
            )
            return True
    facts[goal] = False
    return Fals
# Step 4: Run four consultations
goals = [
    "nitrogen_deficiency",
    "root_rot",
    "leaf_blight",
    "powdery_mildew"
]
for goal in goals
    print("\n" + "=" * 55)
    print("CONSULTATION - Goal:", goal)
```

```
    print("=" * 55)
    facts.clear()
    proof_trace.clear()
    question_trace.clear(
    result = prove(goal
    print("\nHOW: Explanation of conclusion")
    if result:
        print("Goal successfully proved:", goal)
        for rule in proof_trace:
            print("Rule:", rule[0])
            print("Conditions:", rule[1])
            print("Conclusion:", rule[2])
    else:
        print("Goal could not be proved:", goal)
    print("\nQuestions asked:", question_trace)
    print("Number of questions:", len(question_trace))
print("\nEXPERIMENT 5 COMPLETED SUCCESSFULLY!")
```

Output:



Experiment 6: Building and Reasoning over an Ontology

Program:

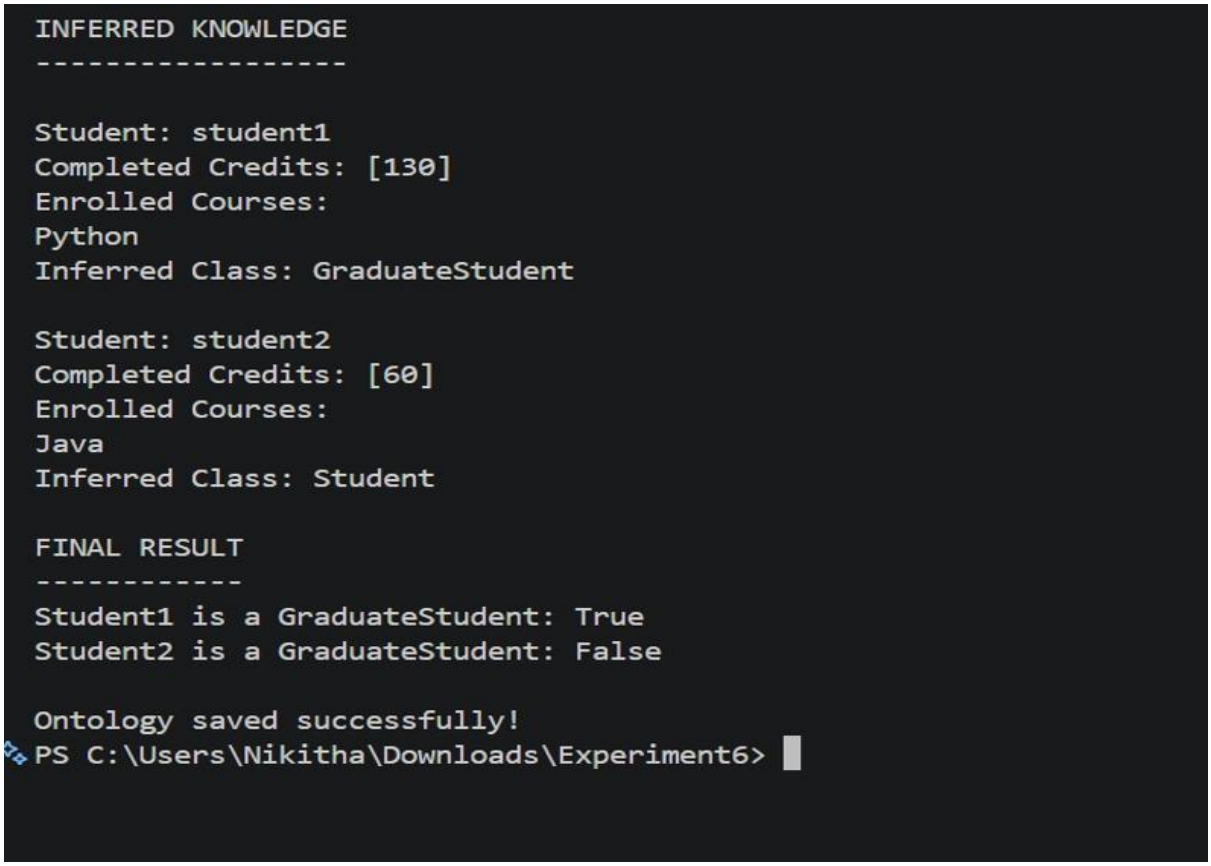
```
# Step 1: Create ontology
onto = get_ontology("http://example.org/university.owl")
with onto:
    # Step 2: Create classes
    class Person(Thing):
        pass
    class Student(Person):
        pass
    class Faculty(Person):
        pass
    class UGStudent(Student):
        pass
    class PGStudent(Student):
        pass
    class Professor(Faculty):
        pass
    class Course(Thing):
        pass
    class Department(Thing):
        pass
    # Step 3: Define disjoint classes
    AllDisjoint([Student, Faculty])
    # Step 4: Create object properties
    class enrolledIn(ObjectProperty):
        domain = [Student]
        range = [Course]
```

```
class teaches(ObjectProperty):
    domain = [Faculty]
    range = [Course]
class belongsTo(ObjectProperty):
    domain = [Person]
    range = [Department]
# Step 5: Create data properties
class hasName(DataProperty):
    domain = [Person]
    range = [str]
class hasCredits(DataProperty):
    domain = [Course]
    range = [int]
# Step 6: Course must have at least one teacher
Course.is_a.append(teaches.some(Faculty))
# Step 7: Create departments
cse = Department("CSE_Department")
ece = Department("ECE_Department")
# Step 8: Create courses
python_course = Course("Python")
ai_course = Course("Artificial_Intelligence")
dbms_course = Course("DBMS")
python_course.hasCredits = [4]
ai_course.hasCredits = [3]
dbms_course.hasCredits = [4]
# Step 9: Create faculty
faculty1 = Professor("Professor_A")
faculty2 = Faculty("Faculty_B")
faculty1.hasName = ["Dr. Kumar"]
faculty2.hasName = ["Dr. Ravi"]
```

```
    faculty1.teaches = [python_course, ai_course]
    faculty2.teaches = [dbms_course]
    # Step 10: Create students
    students = []
    for i in range(1, 11):
        s = Student("Student_" + str(i))
        s.hasName = ["Student " + str(i)]
        s.belongsTo = [cse if i <= 5 else ece]
        s.enrolledIn = [
            python_course if i % 2 == 0 else ai_course
        ]
        students.append(s)
    # Add another course enrollment
    students[0].enrolledIn.append(dbms_course)
# Step 11: Save ontology
onto.save(file="university.owl", format="rdfxml")
print("UNIVERSITY ONTOLOGY CREATED SUCCESSFULLY!")
# Step 12: Display classes
print("\nONTOLOGY CLASSES")
for cls in onto.classes():
    print(cls.name)
# Step 13: Display individuals
print("\nONTOLOGY INDIVIDUALS")
for individual in onto.individuals():
    print(individual.name)
# Step 14: Run reasoner
print("\nRUNNING ONTOLOGY REASONER")
with onto:
    sync_reasoner(infer_property_values=True)
```

```
print("Reasoning completed!")
# Step 15: Display inferred information
print("\nINFERRED CLASS MEMBERSHIPS")
for student in students:
    print(student.name, "is a Student:", Student in student.is_a)
# Step 16: Competency questions
print("\nCOMPETENCY QUESTIONS")
print("\n1. Which courses does Student_1 attend?")
print([c.name for c in students[0].enrolledIn])
print("\n2. Which courses does Professor_A teach?")
print([c.name for c in faculty1.teaches])
print("\n3. Which department does Student_1 belong to?")
print([d.name for d in students[0].belongsTo])
print("\n4. How many students are created?")
print(len(students))
print("\n5. Which courses are available?")
print([c.name for c in onto.Course.instances()])
# Step 17: Deliberately create inconsistency
print("\nTESTING ONTOLOGY INCONSISTENCY")
students[0].is_a.append(Faculty)
try:
    with onto:
        sync_reasoner()
except OwlReadyInconsistentOntologyError:
    print("INCONSISTENCY DETECTED!")
    print("Student_1 is both Student and Faculty.")
    print("These classes are declared disjoint.")
# Step 18: Downloadable ontology
print("\nOntology saved as university.owl")
print("\nEXPERIMENT 6 COMPLETED SUCCESSFULLY!")
```

Output:



Experiment 7: Building and Querying a Knowledge Graph

Program:

```
!pip install pandas rdflib networkx matplotlib -q
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
from rdflib import Graph, Namespace, URIRef, Literal
from rdflib.namespace import RDF, RDFS
from difflib import SequenceMatcher
# Step 1: Create people dataset
people = pd.DataFrame({
    "person_id": [1, 2, 3, 4, 5],
```

```
    "name": ["Asha", "Arun Kumar", "A. Kumar", "Ravi", "Priya"],
    "department": ["CSE", "IT", "IT", "ECE", "CSE"],
    "city": ["Chennai", "Chennai", "Chennai", "Bangalore", "Chennai"]
})

# Step 2: Create works dataset
works = pd.DataFrame({
    "person_id": [1, 2, 3, 4, 5],
    "project": ["Alpha", "Beta", "Gamma", "Alpha", "Beta"],
    "role": ["Developer", "Analyst", "Tester", "Manager", "Designer"]
})

people.to_csv("people.csv", index=False)
works.to_csv("works.csv", index=False)

print("PEOPLE DATA")
print(people)

print("\nWORKS DATA")
print(works)

# Step 3: Create RDF graph and namespace
g = Graph()
EX = Namespace("http://example.org/")

g.bind("ex", EX)

# Step 4: Define schema
classes = ["Person", "Project", "Department", "City"]
```

```
for name in classes:
    g.add((EX[name], RDF.type, RDFS.Class))

properties = ["worksOn", "memberOf", "livesIn", "hasRole"]

for prop in properties:
    g.add((EX[prop], RDF.type, RDF.Property))

# Step 5: Convert CSV records into RDF triples
for _, row in people.iterrows():

    person = EX["Person" + str(row["person_id"])]

    g.add((person, RDF.type, EX.Person))
    g.add((person, RDFS.label, Literal(row["name"])))

    g.add((
        person,
        EX.memberOf,
        EX[row["department"]]
    ))

    g.add((
        person,
        EX.livesIn,
        EX[row["city"].replace(" ", "_")]
    ))

    g.add((EX[row["department"]], RDF.type, EX.Department))
    g.add((EX[row["city"].replace(" ", "_")], RDF.type, EX.City))
```

```
# Step 6: Add project relationships
for _, row in works.iterrows():

    person = EX["Person" + str(row["person_id"])]
    project = EX["Project_" + row["project"]]

    g.add((project, RDF.type, EX.Project))
    g.add((person, EX.worksOn, project))
    g.add((person, EX.hasRole, Literal(row["role"])))

# Step 7: Extract triples from text using patterns
sentences = [
    "Asha works on Project Alpha",
    "Arun Kumar works on Project Beta",
    "Ravi works on Project Alpha",
    "Priya works on Project Beta",
    "Asha works on Project Beta",
    "Ravi works on Project Gamma",
    "Priya works on Project Alpha",
    "Arun Kumar works on Project Gamma",
    "Asha works on Project Gamma",
    "Ravi works on Project Beta"
]
print("\nTEXT-BASED TRIPLE EXTRACTION")
for sentence in sentences:
    parts = sentence.split(" works on Project ")
    if len(parts) == 2:
        person_name, project_name = parts
match = people[people["name"] == person_name]
```

```
        if not match.empty:
            person_id = match.iloc[0]["person_id"]
            person = EX["Person" + str(person_id)]
            project = EX["Project_" + project_name]
            g.add((person, EX.worksOn, project))
            print(sentence)
# Step 8: Entity resolution
print("\nENTITY RESOLUTION")
name1 = "Arun Kumar"
name2 = "A. Kumar"
similarity = SequenceMatcher(
    None, name1.lower(), name2.lower()
).ratio()
department1 = people.loc[
    people["name"] == name1, "department"
].iloc[0]
department2 = people.loc[
    people["name"] == name2, "department"
].iloc[0]

if similarity >= 0.5 and department1 == department2:

    print("Duplicate detected:", name1, "and", name2)

    # Merge duplicate entity into Person2
    g.add((
        EX.Person2,
        RDFS.label,
        Literal("A. Kumar (also known as Arun Kumar)")
    ))
```

```
    g.remove((EX.Person3, None, None))
    for _, predicate, obj in list(g.triples(
        (EX.Person3, None, None)
    )):
        g.add((EX.Person2, predicate, obj))
    for subject, predicate, obj in list(g.triples(
        (None, None, EX.Person3)
    )):
        g.add((subject, predicate, EX.Person2)
    print("Entities merged into Person2"
# Step 9: SPARQL queries
queries = {
    "People working on Project Alpha": """
        PREFIX ex: <http://example.org/>
        SELECT DISTINCT ?person
        WHERE {
            ?person ex:worksOn ex:Project_Alpha .
        }
    """,
    "People in CSE department": """
        PREFIX ex: <http://example.org/>
        SELECT ?person
        WHERE {
            ?person ex:memberOf ex:CSE .
        }
    """,
    "People living in Chennai": """
        PREFIX ex: <http://example.org/>
        SELECT ?person
        WHERE {
```

```
            ?person ex:livesIn ex:Chennai .
        }
    """,

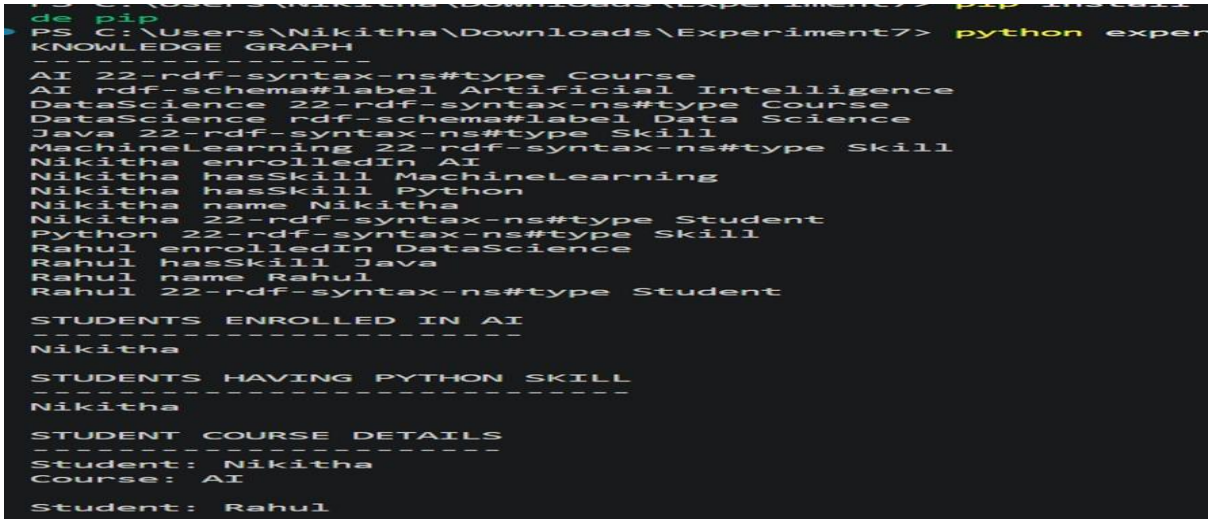
    "People working on Project Beta": """
        PREFIX ex: <http://example.org/>
        SELECT DISTINCT ?person
        WHERE {
            ?person ex:worksOn ex:Project_Beta .
        }
    """,
    "Projects and their workers": """
        PREFIX ex: <http://example.org/>
        SELECT DISTINCT ?person ?project
        WHERE {
            ?person ex:worksOn ?project .
        }
    """
}
print("\nSPARQL QUERY RESULTS"
for title, query in queries.items():
    print("\n", title
    for row in g.query(query):
        print(row
# Step 10: Save knowledge graph
g.serialize(destination="knowledge_graph.ttl", format="turtle"
print("\nKnowledge graph saved as knowledge_graph.ttl")

# Step 11: Convert RDF graph into NetworkX graph
G = nx.Graph()
```

```
for subject, predicate, obj in g:
    if isinstance(subject, URIRef) and isinstance(obj, URIRef)
        G.add_edge(
            subject.split("/")[-1],
            obj.split("/")[-1],
            relation=predicate.split("/")[-1]

# Step 12: Calculate node degrees
print("\nNODE DEGREE")
degrees = dict(G.degree())
for node, degree in degrees.items():
    print(node, ":", degree)
if degrees:
    print("\nMost connected entity:",
          max(degrees, key=degrees.get))
# Step 13: Draw knowledge graph
plt.figure(figsize=(14, 9))
pos = nx.spring_layout(G, seed=42)
nx.draw(
    G, pos,
    with_labels=True,
    node_color="lightgreen",
    node_size=1800,
    font_size=8,
    arrows=False
)
plt.title("Knowledge Graph Visualization")
plt.savefig("knowledge_graph.png", dpi=300, bbox_inches="tight")
plt.show()
print("\nEXPERIMENT 7 COMPLETED SUCCESSFULLY!")
```

Output:



Experiment 8: Hypothesis Testing, ANOVA and Chi-Square with Evidence

Program:

```
!pip install pandas numpy scipy matplotlib -q
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
# Step 1: Generate dataset
np.random.seed(42)
n = 60
methods = np.repeat(["A", "B", "C"], 20)
score_before = np.random.randint(40, 70, n)
improvement = np.where(
    methods == "A", 8,
    np.where(methods == "B", 15, 20)
)
```

```
score_after = np.clip(
    score_before + improvement +
    np.random.randint(-5, 6, n),
    0, 100
)
gender = np.random.choice(
    ["Male", "Female"], n
)
result = np.where(
    score_after >= 60, "Pass", "Fail"
)
df = pd.DataFrame({
    "student": range(1, n + 1),
    "method": methods,
    "score_before": score_before,
    "score_after": score_after,
    "gender": gender,
    "result": result
})
df.to_csv("exam_scores.csv", index=False
print("DATASET")
print(df.head(10))
# Step 2: Significance level
alpha = 0.05
results = []
def report_test(name, statistic, pvalue, h0, h1):
    decision = (
        "Reject H0"
        if pvalue < alpha
        else "Fail to reject H0"    )
```

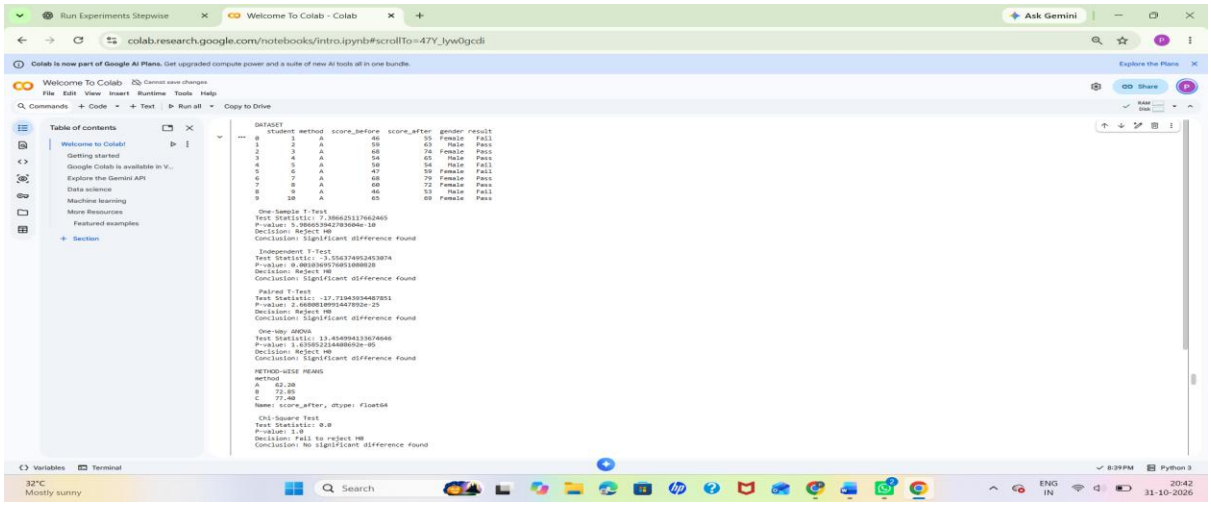
```
    conclusion = (
        "Significant difference found"
        if pvalue < alpha
        else "No significant difference found"
    )
    results.append({
        "Test": name,
        "H0": h0,
        "H1": h1,
        "Statistic": statistic,
        "P-value": pvalue,
        "Decision": decision,
        "Conclusion": conclusion
    })
    print("\n", name)
    print("Test Statistic:", statistic)
    print("P-value:", pvalue)
    print("Decision:", decision)
    print("Conclusion:", conclusion)
# Step 3: One-sample t-test
t1, p1 = stats.ttest_1samp(
    df["score_after"], 60
)
report_test(
    "One-Sample T-Test",
    t1, p1,
    "Mean score is 60",
    "Mean score is not 60"
)
```

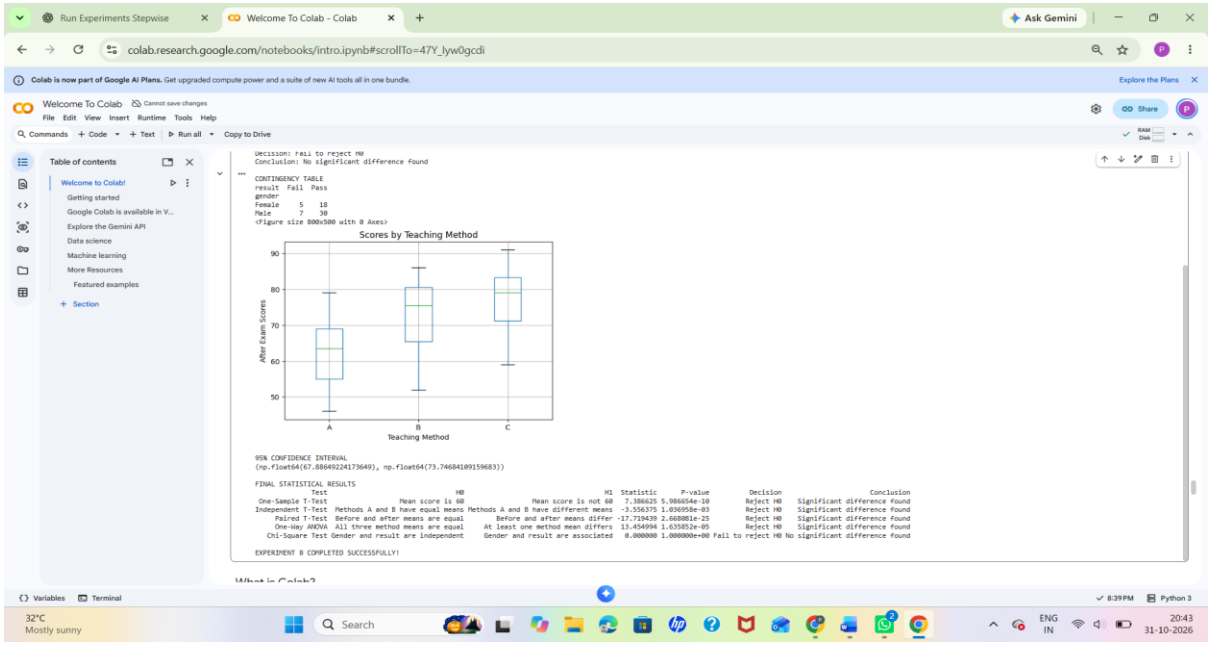
```
# Step 4: Independent-samples t-test
group_a = df[df["method"] == "A"]["score_after"]
group_b = df[df["method"] == "B"]["score_after"]
t2, p2 = stats.ttest_ind(
    group_a, group_b,
    equal_var=False
)
report_test(
    "Independent T-Test",
    t2, p2,
    "Methods A and B have equal means",
    "Methods A and B have different means"
)
# Step 5: Paired t-test
t3, p3 = stats.ttest_rel(
    df["score_before"],
    df["score_after"]
)
report_test(
    "Paired T-Test",
    t3, p3,
    "Before and after means are equal",
    "Before and after means differ"
)
# Step 6: One-way ANOVA
a = df[df["method"] == "A"]["score_after"]
b = df[df["method"] == "B"]["score_after"]
c = df[df["method"] == "C"]["score_after"]
f_stat, p4 = stats.f_oneway(a, b, c)
```

```
report_test(
    "One-Way ANOVA",
    f_stat, p4,
    "All three method means are equal",
    "At least one method mean differs"
)
print("\nMETHOD-WISE MEANS")
print(df.groupby("method")["score_after"].mean())
# Step 7: Chi-square test
contingency = pd.crosstab(
    df["gender"],
    df["result"]
)
chi2, p5, dof, expected = stats.chi2_contingency(
    contingency
)
report_test(
    "Chi-Square Test",
    chi2, p5,
    "Gender and result are independent",
    "Gender and result are associated"
)
print("\nCONTINGENCY TABLE")
print(contingency)
# Step 8: Boxplot
plt.figure(figsize=(8, 5))
df.boxplot(
    column="score_after",
    by="method"
)
```

```
plt.title("Scores by Teaching Method")
plt.suptitle("")
plt.xlabel("Teaching Method")
plt.ylabel("After Exam Scores")
plt.show()
# Step 9: 95% Confidence interval
mean = df["score_after"].mean()
sem = stats.sem(df["score_after"])
confidence_interval = stats.t.interval(
    0.95,
    df=len(df)-1,
    loc=mean,
    scale=sem
)
print("\n95% CONFIDENCE INTERVAL")
print(confidence_interval)
# Step 10: Final results table
print("\nFINAL STATISTICAL RESULTS")
results_df = pd.DataFrame(results)
print(results_df.to_string(index=False))
results_df.to_csv(
    "hypothesis_test_results.csv",
    index=False
)
print("\nEXPERIMENT 8 COMPLETED SUCCESSFULLY!")
```

Output:





Experiment 9: Acquiring Rules from Data with Decision Trees

Program:

```
!pip install pandas numpy matplotlib scikit-learn -q
import numpy as npimport pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import (
    train_test_split,
```

```
    cross_val_score
)
from sklearn.tree import (
    DecisionTreeClassifier,
    export_text
)
from sklearn.dummy import DummyClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    precision_score,
    recall_score,
    f1_score
)
# Step 1: Load dataset
data = load_breast_cancer()
X = pd.DataFrame(
    data.data,
    columns=data.feature_names
)
y = pd.Series(data.target)
print("DATASET INFORMATION")
print("Total Records:", len(X))
print("Total Features:", X.shape[1])
print("Target Classes:", data.target_names)
print("\nFIRST FIVE RECORDS")
print(X.head())
# Step 2: Split dataset (70:30)
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y,
    test_size=0.30,
    random_state=42,
    stratify=y
)
# Step 3: Baseline model
baseline = DummyClassifier(
    strategy="most_frequent"
)
baseline.fit(X_train, y_train)
baseline_pred = baseline.predict(X_test
baseline_accuracy = accuracy_score(
    y_test, baseline_pred
)
print("\nBASELINE ACCURACY:", baseline_accuracy)
# Step 4: Expert-style rules
def expert_rules(X):
    predictions = []
    for _, row in X.iterrows():
        # Rule 1 and Rule 2
        if (
            row["mean concave points"] > 0.05
            or row["worst perimeter"] > 115
        ):
            predictions.append(0)

        # Rule 3
        elif (
            row["mean radius"] > 15
            and row["mean concavity"] > 0.10
```

```
        ):
            predictions.append(0)
        # Rule 4
        elif (
            row["worst area"] > 900
        ):
            predictions.append(0)
        else:
            predictions.append(1)
    return np.array(predictions
expert_pred = expert_rules(X_test
expert_accuracy = accuracy_score(
    y_test, expert_pred
)
print("\nEXPERT RULE ACCURACY:", expert_accuracy)
# Step 5: Train decision trees with different depths
depths = [2, 3, 4, 6]
training_accuracy = []
testing_accuracy = []
cv_accuracy = []
models = {}
print("\nDECISION TREE RESULTS")
for depth in depths
    model = DecisionTreeClassifier(
        max_depth=depth,
        random_state=42

    model.fit(X_train, y_train)

    train_pred = model.predict(X_train)
```

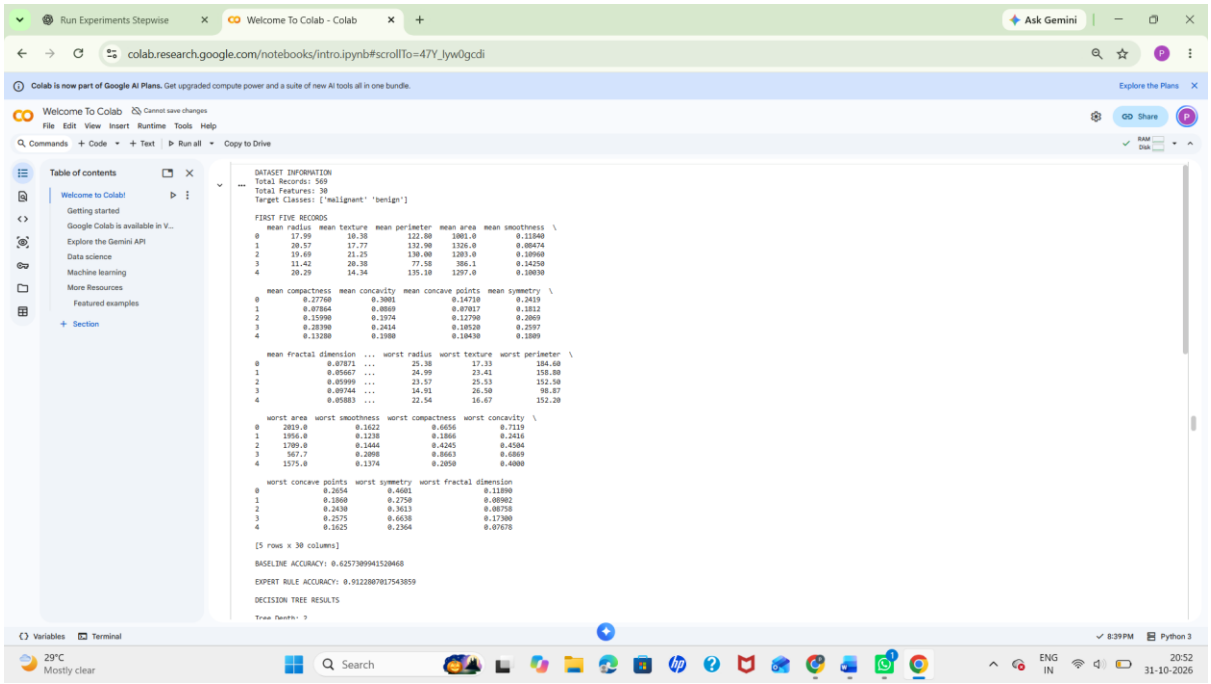
```
    test_pred = model.predict(X_test)
    train_acc = accuracy_score(
        y_train, train_pred
    )
    test_acc = accuracy_score(
        y_test, test_pred
    )
    cv_scores = cross_val_score(
        model,
        X_train,
        y_train,
        cv=5,
        scoring="accuracy"
    )
    training_accuracy.append(train_acc)
    testing_accuracy.append(test_acc)
    cv_accuracy.append(cv_scores.mean())
    models[depth] = model
    print("\nTree Depth:", depth)
    print("Training Accuracy:", train_acc)
    print("Testing Accuracy:", test_acc)
    print("5-Fold CV Accuracy:", cv_scores.mean())
# Step 6: Accuracy comparison table
comparison = pd.DataFrame({
    "Depth": depths,
    "Training Accuracy": training_accuracy,
    "Testing Accuracy": testing_accuracy,
    "Cross Validation": cv_accuracy
})
print("\nCOMPARISON TABLE")
```

```
print(comparison.to_string(index=False))
# Step 7: Plot accuracy against depth
plt.figure(figsize=(8, 5))
plt.plot(
    depths,
    training_accuracy,
    marker="o",
    label="Training Accuracy"
)
plt.lot(
    depths,
    testing_accuracy,
    marker="s",
    label="Testing Accuracy"
plt.plot(
    depths,
    cv_accuracy,
    marker="^",
    label="Cross Validation"
)
plt.xlabel("Tree Depth")
plt.ylabel("Accuracy")
plt.title("Accuracy vs Decision Tree Depth")
plt.legend()
plt.grid()
plt.show()
# Step 8: Select best depth using CV
best_depth = depths[np.argmax(cv_accuracy)]
best_model = models[best_depth]
print("\nBEST TREE DEPTH:", best_depth)
```

```
# Step 9: Display decision tree rules
tree_rules = export_text(
    best_model,
    feature_names=list(X.columns)
)
print("\nLEARNED DECISION TREE RULES")
print(tree_rules)
# Step 10: Evaluate best model
tree_pred = best_model.predict(X_test)
print("\nCONFUSION MATRIX")
print(confusion_matrix(y_test, tree_pred))
print("\nCLASSIFICATION REPORT")
print(
    classification_report(
        y_test,
        tree_pred,
        target_names=data.target_names
    )
)
# Step 11: Compare expert rules and decision tree
print("\nMODEL COMPARISON")
for name, pred in [
    ("Expert Rules", expert_pred),
    ("Decision Tree", tree_pred)
]:
    print("\n", name)
    print("Accuracy:",
          accuracy_score(y_test, pred))
    print("Precision:",
          precision_score(y_test, pred, zero_division=0))
```

```
    print("Recall:",
          recall_score(y_test, pred, zero_division=0))
    print("F1 Score:",
          f1_score(y_test, pred, zero_division=0))
# Step 12: Save results
comparison.to_csv(
    "decision_tree_comparison.csv",
    index=False
)
print("\nEXPERIMENT 9 COMPLETED SUCCESSFULLY!")
```

Output:



Experiment 10: Hybrid Decision-Support System (Knowledge Graph, Rules and Machine Learning)

Program:

```
!pip install pandas numpy networkx matplotlib scikit-learn -q
import pandas as pd
import numpy as np
```

```
import networkx as nx
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import (
    train_test_split,
    cross_val_score
)
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    f1_score,
    classification_report
)
# STEP 1: Generate student dataset
np.random.seed(42)
n = 12
departments = ["CSE", "IT", "ECE"]
interests = ["AI", "Web", "Data"]
python_skills = ["High", "Medium", "Low"]
math_skills = ["High", "Medium", "Low"]

df = pd.DataFrame({
    "student_id": range(1, n + 1),
    "department": np.random.choice(departments, n),
    "interest": np.random.choice(interests, n),
    "python_skill": np.random.choice(python_skills, n),
    "math_skill": np.random.choice(math_skills, n)
})
```

```
# Generate target course based on student information
def assign_course(row):
    if row["interest"] == "AI":
        return "Machine_Learning"
    elif row["interest"] == "Web":
        return "Web_Development"
    else:
        return "Data_Science"
df["preferred_course"] = df.apply(
    assign_course, axis=1
)
df.to_csv("student_courses.csv", index=False)
print("STUDENT DATASET")
print(df.head(10))
# STEP 2: Create Knowledge Graph
KG = nx.DiGraph(
# Create student, course, skill and department nodes
for _, row in df.iterrows():
    student = "Student_" + str(row["student_id"])
    KG.add_edge(
        student,
        row["department"],
        relation="belongsTo"
    )
    KG.add_edge(
        student,
        row["interest"],
        relation="interestedIn"
    )
```

```
    KG.add_edge(
        student,
        row["preferred_course"],
        relation="recommendedCourse"
    )
# Add course relationships
course_skills = {
    "Machine_Learning": ["Python", "Mathematics"],
    "Web_Development": ["HTML", "JavaScript"],
    "Data_Science": ["Python", "Statistics"]
}
for course, skills in course_skills.items():
    KG.add_node(course, type="Course")
    for skill in skills:
        KG.add_edge(
            course,
            skill,
            relation="requiresSkill"

print("\nKNOWLEDGE GRAPH CREATED")
print("Total Triples:", KG.number_of_edges())
# STEP 3: Define 10 expert rules with certainty factors
rules = [
    ("R1", "AI", "Machine_Learning", 0.95),
    ("R2", "Web", "Web_Development", 0.95),
    ("R3", "Data", "Data_Science", 0.95),
    ("R4", "High_Python", "Machine_Learning", 0.80),
    ("R5", "High_Math", "Machine_Learning", 0.85),
    ("R6", "High_Python", "Data_Science", 0.80),
    ("R7", "High_Web", "Web_Development", 0.90),
```

```
    ("R8", "Low_Python", "Machine_Learning", -0.50),
    ("R9", "High_Math", "Data_Science", 0.75),
    ("R10", "Low_Math", "Machine_Learning", -0.40)
]

print("\nEXPERT RULES")
for rule in rules:
    print(rule)

# STEP 4: Rule-based prediction
def rule_prediction(row):

    scores = {
        "Machine_Learning": 0,
        "Web_Development": 0,
        "Data_Science": 0
    }
    for rule_id, condition, course, cf in rules
        matched = False
        if condition == row["interest"]:
            matched = True
        elif condition == "High_Python":
            matched = row["python_skill"] == "High
        elif condition == "High_Math":
            matched = row["math_skill"] == "High"
        elif condition == "Low_Python":
            matched = row["python_skill"] == "Low"
        elif condition == "Low_Math":
            matched = row["math_skill"] == "Low
        elif condition == "High_Web":
```

```
            matched = row["interest"] == "Web"
        if matched:
            scores[course] += cf
    return max(scores, key=scores.ge
df["rule_prediction"] = df.apply(
    rule_prediction, axis=1
)
# STEP 5: Prepare machine learning dataset
encoders = {
X = df[
    ["department", "interest", "python_skill", "math_skill"]
].copy()
for col in X.columns:
    encoder = LabelEncoder()
    X[col] = encoder.fit_transform(X[col])
    encoders[col] = encoder
target_encoder = LabelEncoder()
y = target_encoder.fit_transform(
    df["preferred_course"]
)
# STEP 6: Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

# STEP 7: Train Random Forest model
model = RandomForestClassifier(
```

```
    n_estimators=100,
    random_state=42
)
model.fit(X_train, y_train)
# Cross-validation
cv_scores = cross_val_score(
    model, X_train, y_train, cv=5
)
print("\nMACHINE LEARNING RESULTS")
print("Cross Validation Accuracy:", cv_scores.mean())

# STEP 8: Model predictions
ml_predictions = model.predict(X_test)

ml_courses = target_encoder.inverse_transform(
    ml_predictions
)

# STEP 9: Hybrid combination
# Combine rule-based and ML predictions

hybrid_predictions = []
for index, prediction in enumerate(ml_courses):
    original_index = X_test.index[index]
    row = df.loc[original_index]
    rule_result = rule_prediction(row)
    # If the rule and model agree, use their result
    if rule_result == prediction:
        hybrid_result = prediction
```

```
    # If they disagree, use the ML prediction
    else:
        hybrid_result = prediction

    hybrid_predictions.append(hybrid_result)

# STEP 10: Create baseline predictions
baseline_prediction = df.loc[
    X_train.index, "preferred_course"
].mode()[0]

baseline_predictions = [
    baseline_prediction
] * len(y_test)

# STEP 11: Evaluate four systems
actual = target_encoder.inverse_transform(y_test)

rule_predictions = df.loc[
    X_test.index, "rule_prediction"
].values

results = {
    "Baseline": baseline_predictions,
    "Rules Alone": rule_predictions,
    "ML Alone": ml_courses,
    "Hybrid System": hybrid_predictions
}

evaluation = []
```

```
print("\nSYSTEM EVALUATION")

for system, predictions in results.items():

    accuracy = accuracy_score(
        actual, predictions
    )

    precision = precision_score(
        actual, predictions,
        average="weighted",
        zero_division=0
    )

    f1 = f1_score(
        actual, predictions,
        average="weighted",
        zero_division=0
    )

    evaluation.append({
        "System": system,
        "Accuracy": accuracy,
        "Precision": precision,
        "F1 Score": f1
    })

    print("\n", system)
    print("Accuracy:", accuracy)
```

```
    print("Precision:", precision)
    print("F1 Score:", f1)

# STEP 12: Display comparison table
evaluation_df = pd.DataFrame(evaluation)
print("\nFINAL COMPARISON TABLE")
print(evaluation_df.to_string(index=False))
# STEP 13: Test 20 cases
print("\n20 SAMPLE TEST CASES")
sample = X_test.iloc[:20]
sample_actual = actual[:20]
sample_hybrid = hybrid_predictions[:20]
fori in range(20):
    print(
        "Case", i + 1,
        "| Actual:", sample_actual[i],
        "| Predicted:", sample_hybrid[i]
    )
# STEP 14: Display explanation
print("\nSAMPLE RECOMMENDATION EXPLANATIONS")
for i in range(5):
    index = X_test.index[i]
    row = df.loc[index]
    print("\nStudent:", row["student_id"])
    print("Interest:", row["interest"])
    print("Python Skill:", row["python_skill"])
    print("Math Skill:", row["math_skill"])
    print("Knowledge Graph Facts:")
    student = "Student_" + str(row["student_id"])
    print(list(KG.edges(student)))
```

```
    print("Rule-Based Recommendation:",
          rule_prediction(row))
    print("ML Recommendation:",
          ml_courses[i])
    print("Hybrid Recommendation:",
          hybrid_predictions[i])
    probabilities = model.predict_proba(
        X_test.iloc[[i]]
    )[0]
    print(
        "ML Confidence:",
        round(float(max(probabilities)), 3)
    )
# STEP 15: Visualize knowledge graph
plt.figure(figsize=(12, 8))
pos = nx.spring_layout(KG, seed=42)
nx.draw(
    KG,
    pos,
    with_labels=True,
    node_color="lightblue",
    node_size=900,
    font_size=7,
    arrows=True
)
plt.title("Hybrid Decision Support Knowledge Graph")
plt.show()
# STEP 16: Save outputs
evaluation_df.to_csv(
    "hybrid_evaluation.csv",
```

```
    index=False
)
df.to_csv(
    "hybrid_student_results.csv",
    index=False
)
print("\nEXPERIMENT 10 COMPLETED SUCCESSFULLY!")
```

Output:

